



DECISIONROOM G3

STACK TECNOLÓGICO

Frontend: Electron integrado con Python

Backend: Ruby on Rails

Persistencia: Redis

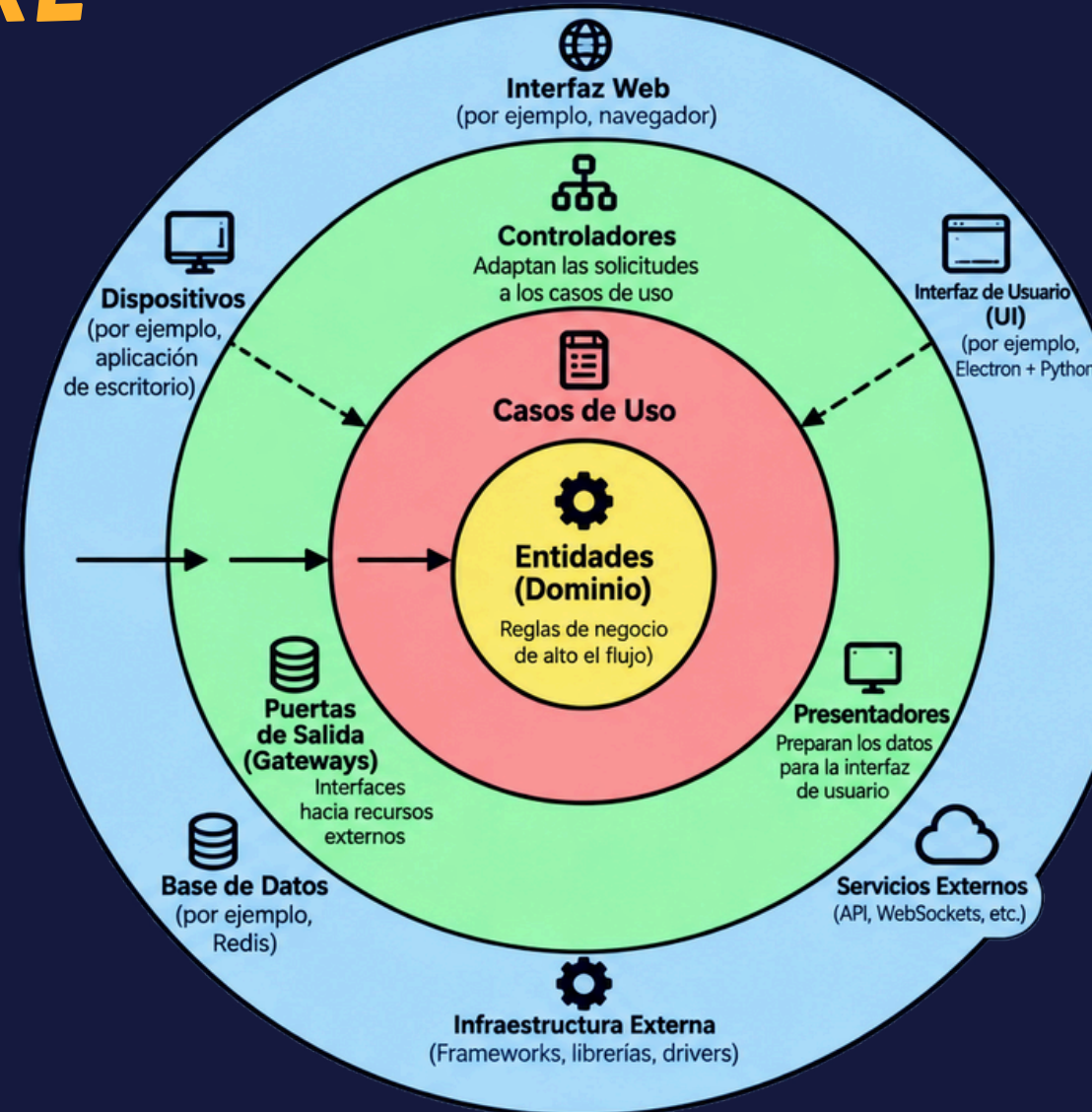
Integración: WebSockets

Infraestructura: Docker compose

PRESENTACIÓN 1 - GRUPO 3

CLEAN ARCHITECTURE

Arquitectura Limpia (Clean Architecture)

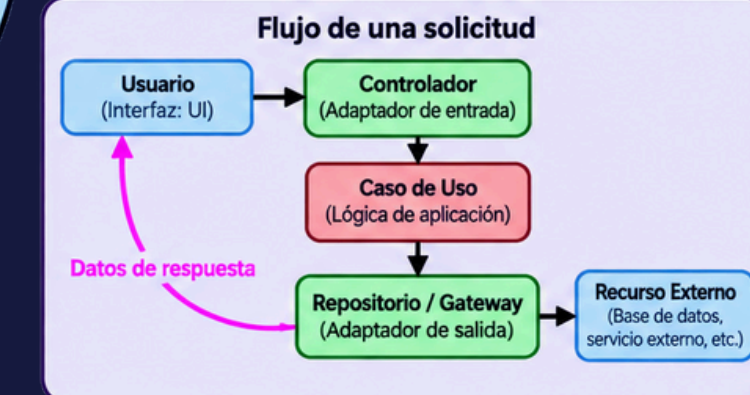


Entidades (Dominio)
Contiene las reglas de negocio más importantes.
No depende de nada externo.

Casos de Uso
Implementa la lógica de la aplicación.
Orquesta el flujo de datos entre entidades y adaptadores.

Adaptadores de Interfaz
Convierte datos entre el formato externo (UI, base de datos, API) y el formato interno de los casos de uso.

Frameworks y Controladores (Infraestructura)
Son detalles de implementación: UI, base de datos, frameworks, drivers, dispositivos, etc.
Pueden cambiar sin afectar el dominio ni los casos de uso.



No es una librería ni una plantilla fija de carpetas: es una regla sobre dependencias.

No es un patrón de clase como Singleton; actúa sobre todo el sistema.

No es capas tradicionales: ahí la lógica de negocio depende de la base de datos.

Círculos concéntricos cuyas dependencias apuntan siempre hacia el centro: la Regla de Dependencia.

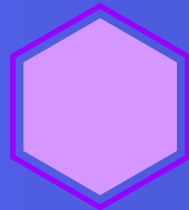
Síntesis de Hexagonal (2005) y Onion (2008); Robert C. Martin lo formalizó en 2012.

El núcleo define un puerto propio y la infraestructura lo implementa (Inversión de Dependencias).

HISTORIA Y EVOLUCIÓN



De la arquitectura hexagonal a la síntesis de Robert C. Martin: veinte años organizando el código alrededor del negocio.



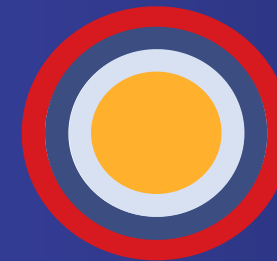
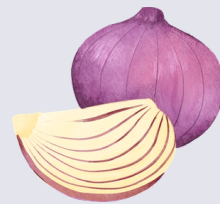
2005

Alistair Cockburn propone separar el núcleo del sistema de elementos externos mediante puertos y adaptadores (hexagonal)

Jeffrey Palermo organiza la arquitectura en capas alrededor del dominio, haciendo que las dependencias apunten hacia el centro (Onion)



2008



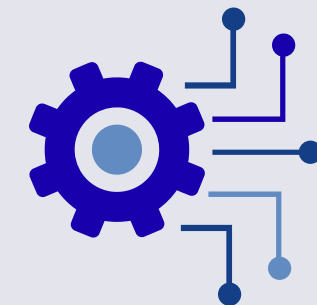
2012

Robert C. Martin reúne estas ideas y propone una arquitectura en 4 capas que es independiente de frameworks, interfaces y bases de datos.

Clean Architecture se populariza como enfoque para construir sistemas mantenibles, testeables y fáciles de adaptar a nuevas tecnologías.



2017



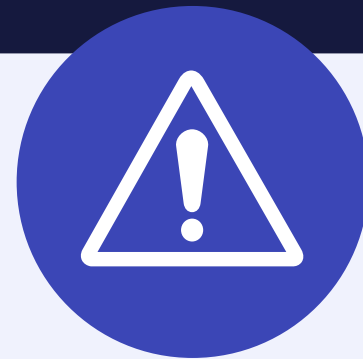
VENTAJAS Y DESVENTAJAS

Aislar el negocio cuesta código extra al inicio, pero paga en mantenibilidad y en pruebas.



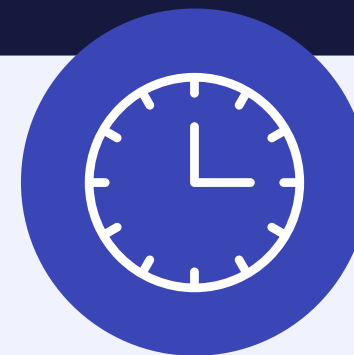
VENTAJA

Los cambios en interfaz o persistencia quedan aislados en los círculos externos.



DESVENTAJA

Exige interfaces, adaptadores y DTOs: más código antes de ver resultados.



CUÁNDO USARLO

Negocio complejo, varios canales de entrada y una vida útil larga.



CASO REAL

eShopOnWeb, de Microsoft, organiza su código con este estilo.

ELECTRON + PYTHON (EEL)

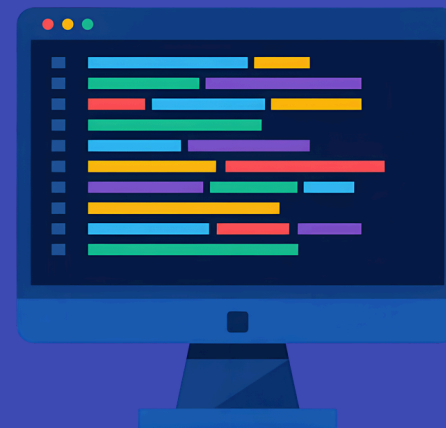


Electron integra Chromium y Node.js para crear apps de escritorio con HTML, CSS y JS.



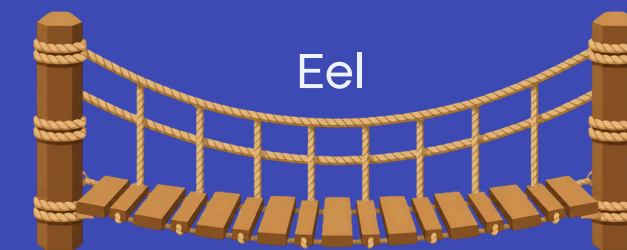
Electron le permite mantener una base de código JavaScript y crear aplicaciones multiplataforma que funcionan en Windows, macOS y Linux

Eel es una librería de Python para crear aplicaciones sencillas sin conexión.



Permite la comunicación bidireccional entre JavaScript y Python, conectando las acciones de la interfaz con la lógica local.

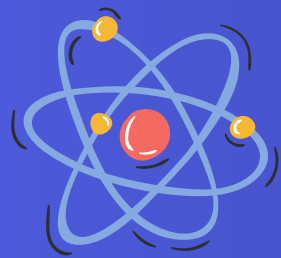
Electron se encarga de ejecutar y mostrar la interfaz gráfica de escritorio



Python se encarga de la lógica y procesamiento local de la aplicación.

HISTORIA Y EVOLUCIÓN

Electron nació para el editor Atom de GitHub; Eel ofrece ese mismo patrón sin salir de Python.



2013

Nace como Atom Shell, proyecto interno de GitHub para el editor Atom.



2014

Atom Shell sale en beta pública y lo adoptan Slack, GitKraken y WebTorrent.



2015

El proyecto se renombra Electron; su titularidad pasa luego a la OpenJS Foundation.



2016

Electron 1.0 promete estabilidad de API; Eel surge como alternativa ligera en Python.

VENTAJAS Y DESVENTAJAS

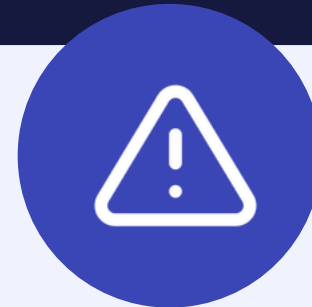


Sin casos documentados de Eel en la industria;
Electron sí los tiene, y de sobra.



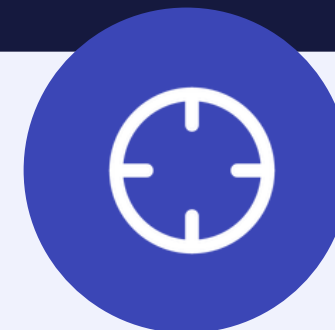
VENTAJA

Interfaz moderna en HTML y CSS
mientras la lógica vive en Python.



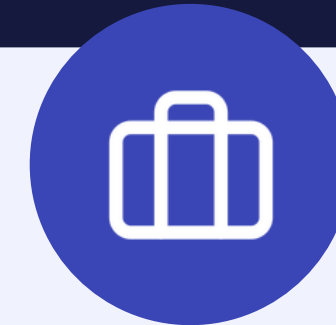
DESVENTAJA

Depende de un navegador
Chromium instalado y usa
monkey patching de Gevent.



CUÁNDO USARLO

Herramientas internas y
utilidades con una interfaz
sencilla.



CASO REAL

Electron: VS Code, Slack y
Discord. Eel: sin caso
documentado.

REDIS



IN-MEMORY DB

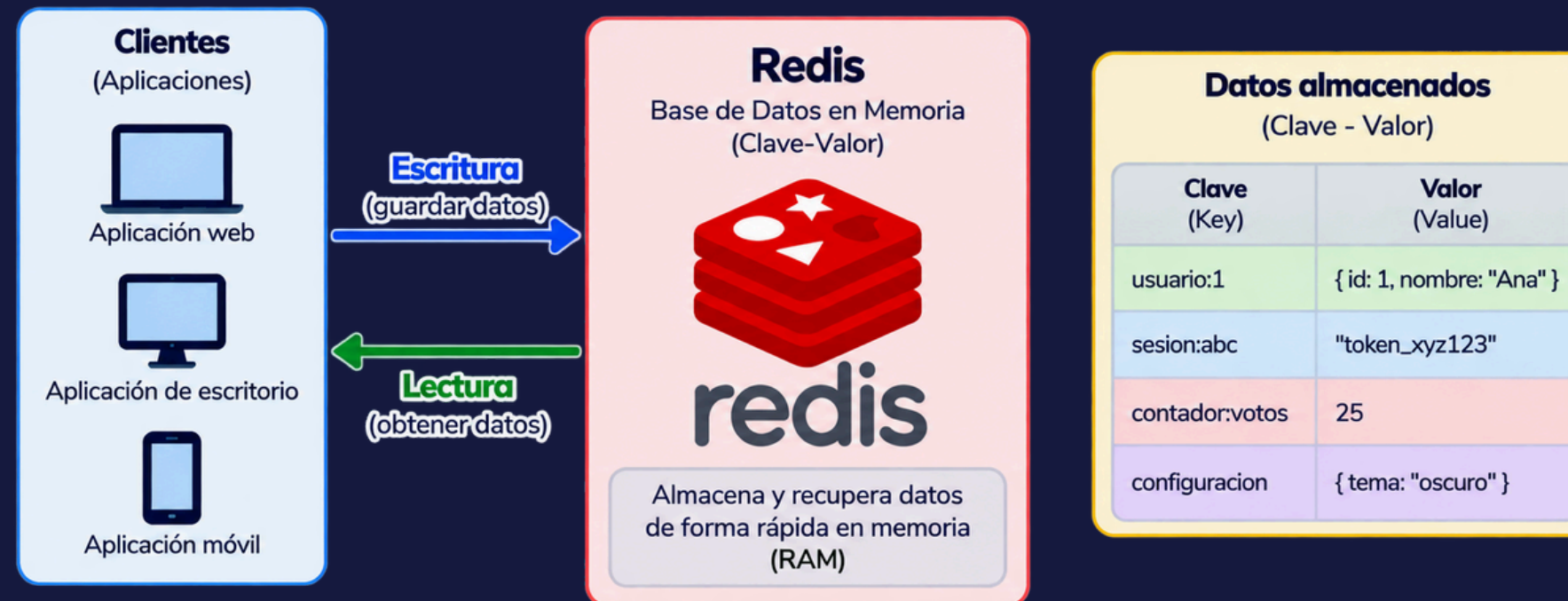


Almacén de estructuras de datos en memoria: caché, base de datos y bróker de mensajes.

Soporta hashes, sets y operaciones atómicas como HINCRBY y SADD.

El repositorio lo usa para contar votos y verificar votantes únicos.

Redis como Base de Datos



No es una base relacional: no ofrece SQL ni uniones nativas entre tablas.

No garantiza durabilidad total; depende de la persistencia RDB o AOF configurada.

No sustituye a un almacén histórico para volúmenes que excedan la memoria.

HISTORIA Y EVOLUCIÓN



De un prototipo italiano de 2009 a una de las bases de datos en memoria más usadas del mercado.

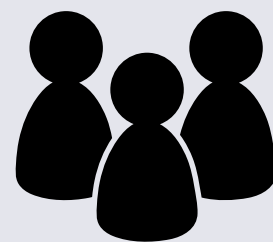


2009

Salvatore Sanfilippo crea Redis para un analizador de registros en tiempo real.

Sanfilippo deja de ser el mantenedor único del proyecto.

2020

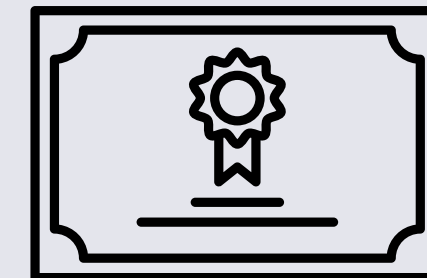


2024

Redis cambia a licencia SSPL/RSAL; la Linux Foundation bifurca Valkey en BSD.

Redis 8 añade AGPLv3 como tercera opción de licencia.

2025



VENTAJAS *Y DESVENTAJAS*

Rapidísimo en memoria, pero el modelado de claves recae por completo en el equipo.



VENTAJA

Operaciones atómicas: ideales para contadores y unicidad concurrente.



DESVENTAJA

El tamaño de los datos queda limitado por la memoria RAM disponible.



CUÁNDO USARLO

Cachés, sesiones, colas y tableros de puntuación en tiempo real.



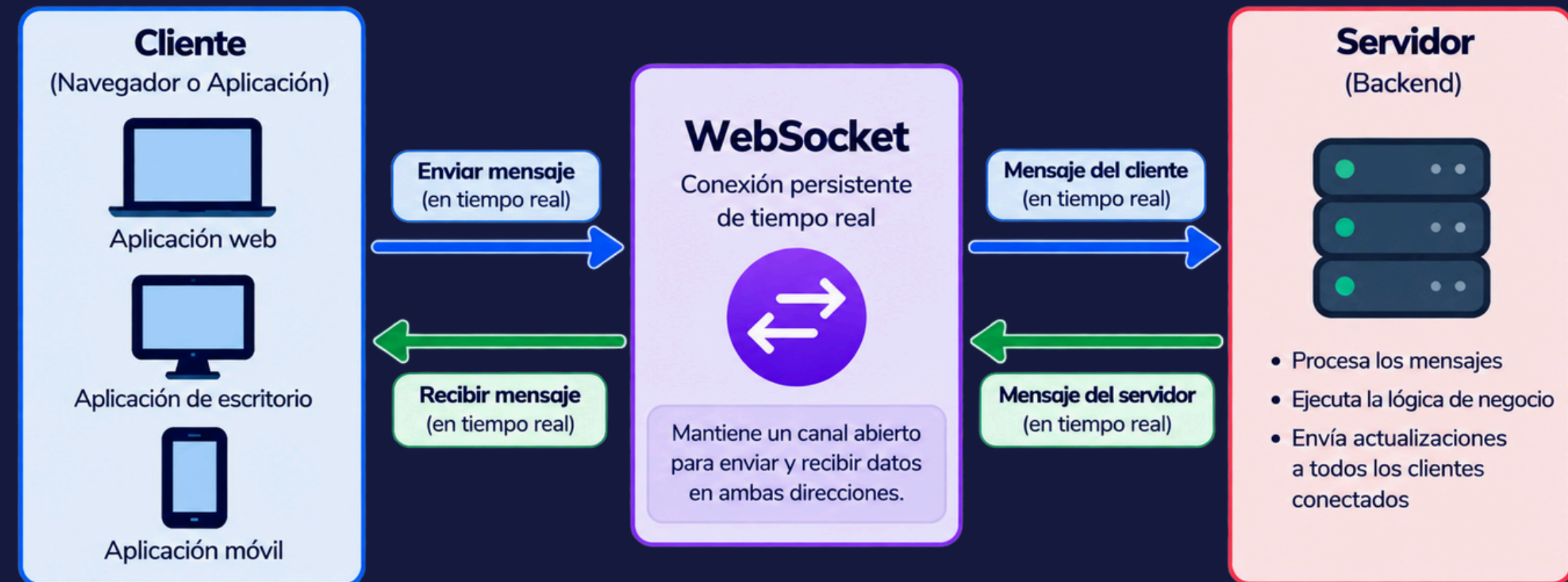
CASO REAL

Twitter, GitHub, Instagram y Slack lo documentan en su ingeniería.

PROTOCOLO WEBSOCKETS



Funcionamiento de WebSocket



Action Cable lo usa para difundir resultados a todos los delegados conectados.

El saludo inicial usa HTTP y luego conmuta al protocolo WebSocket.

Protocolo IETF (RFC 6455, 2011): un canal bidireccional sobre una sola conexión TCP.

No garantiza por sí solo la entrega a quien se reconecta tarde.

No conviene para consultas esporádicas o reportes por lotes.

HISTORIA Y EVOLUCIÓN

De un parche sobre HTTP a un protocolo propio: así llegó la web al tiempo real.



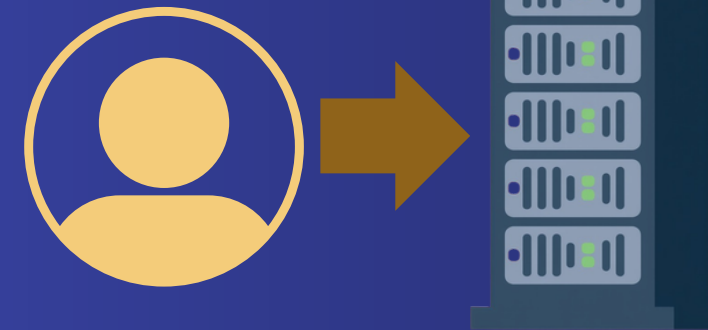
2008

Ian Hickson lo propone dentro de la especificación HTML5.



2010

La tecnología deja de tratarse simplemente como una característica de HTML5 y comienza a desarrollarse como un protocolo independiente.



2011

La IETF publica el RFC 6455, que define oficialmente el protocolo WebSocket y establece cómo deben comunicarse clientes y servidores.



2012

Los principales navegadores incorporan soporte para WebSocket, facilitando su uso en aplicaciones web con comunicación en tiempo real.

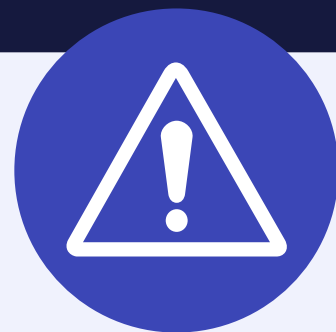
VENTAJAS Y DESVENTAJAS

Ideal para eventos en vivo; exige gestionar el estado de cada conexión.



VENTAJA

Baja latencia: el servidor empuja datos sin que el cliente los pida.



DESVENTAJA

Conexiones persistentes con estado que complican el escalado horizontal.



CUÁNDO USARLO

Chats, juegos en línea, votaciones y tableros de resultados en vivo.



CASO REAL

El Gateway de Discord y el Socket Mode de Slack lo documentan.

MERCADO LABORAL

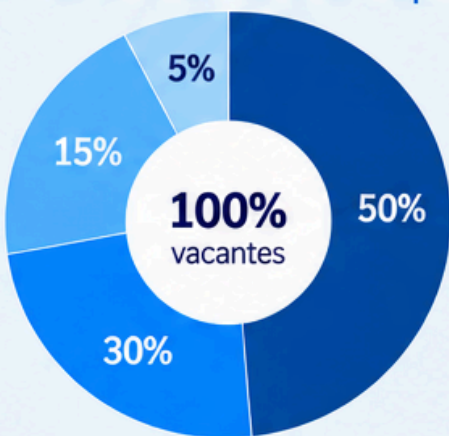
(Proyección y Salarios)

Arquitectura de Software (Clean Arch)

Distribución de vacantes por perfil



Rango salarial anual
\$128,000 - \$211,000+ USD



Senior (Arquitecto)	50%
Staff Engineer	30%
Lead	15%
Otros	5%



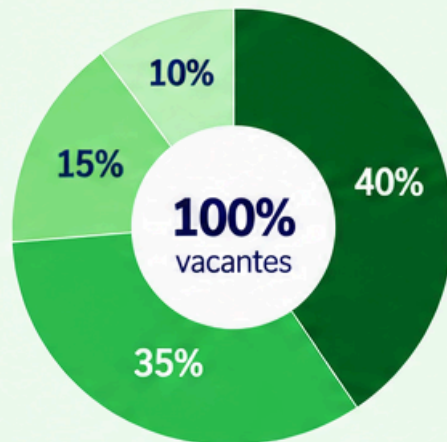
Habilidad que catapulta a los roles de mayor liderazgo en la industria.

Redis (Infraestructura / Datos)

Distribución de vacantes por perfil



Rango salarial anual
\$140,000 - \$154,000 USD



Administrador / DevOps	40%
Ingeniero de Datos	35%
Ingeniero de Infraestructura	15%
Otros	10%



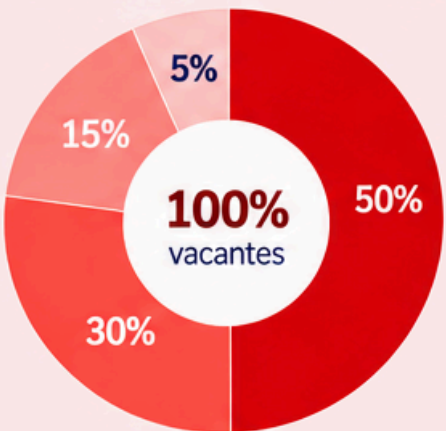
Habilidad premium para escalar sistemas críticos.

Ruby on Rails (Ecosistema SaaS)

Distribución de vacantes por perfil



Rango salarial anual
\$96,000 - \$146,000+ USD



Backend Developer	50%
Full Stack Developer	30%
Software Engineer (Mid / Senior)	15%
Otros	5%



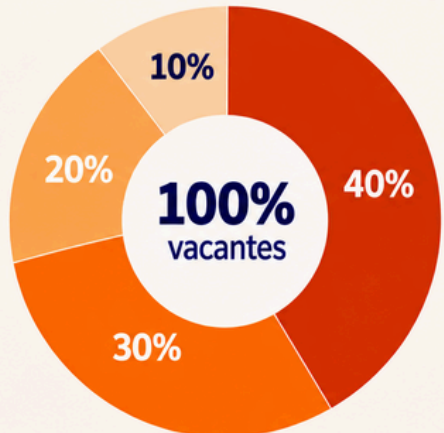
Nicho extremadamente rentable. Muy bien pagado por el mantenimiento de SaaS gigantes (Shopify, GitHub, Stripe).

Electron (Frontend/Desktop)

Distribución de vacantes por perfil



Rango salarial anual
\$52,000 - \$138,000+ USD



Frontend / UI (Electron)	40%
Desktop / Electron Engineer	30%
Full Stack JS/TS	20%
Otros perfiles técnicos	10%



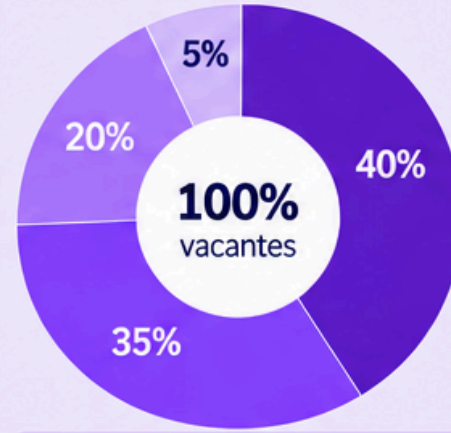
Nicho especializado: combina frontend web con desarrollo de escritorio multiplataforma. Electron suele aparecer junto a JavaScript/TypeScript, Node.js, React, APIs, testing y empaquetado.

Python (Datos e IA)

Distribución de vacantes por perfil



Rango salarial anual
\$96,000 - \$133,000+ USD



Data Engineer	40%
ML / IA Engineer	35%
Data Scientist	20%
Otros	5%



Demanda masiva global. Los roles pueden superar los \$200k USD si se especializan en IA o Datos.

NUESTRA PROPUESTA

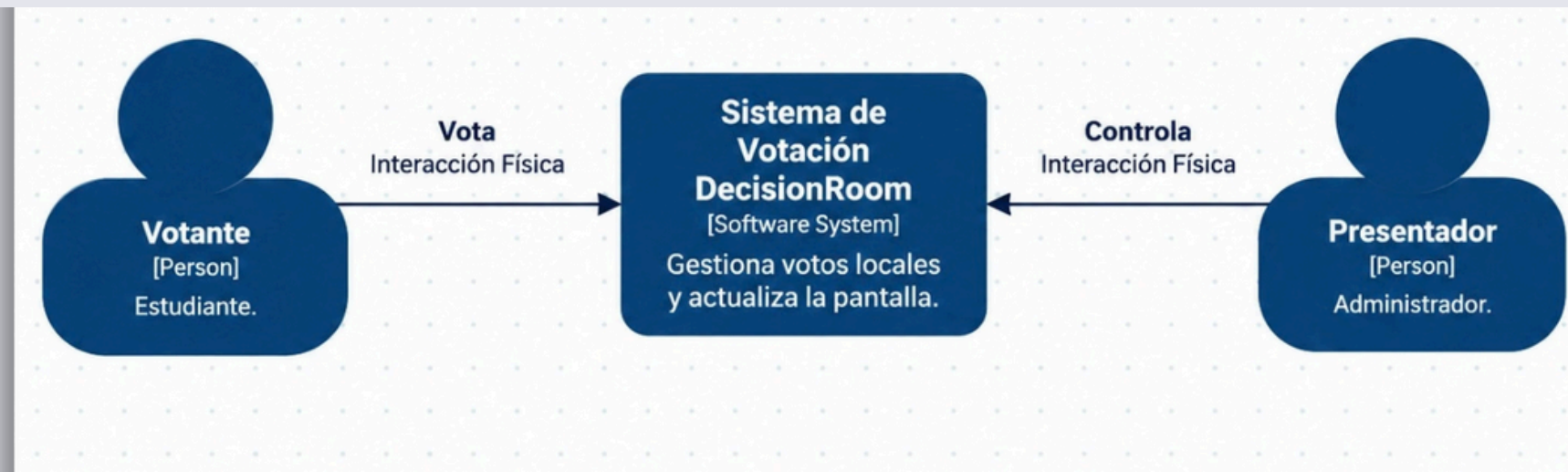
Sincronización en Tiempo Real

Diagrama de Alto Nivel del Sistema de Votación

(Arquitectura general y flujo principal de comunicación)



CONTEXTO *DEL SISTEMA*



COMO FUNCIONAN *LOS CONTENEDORES*

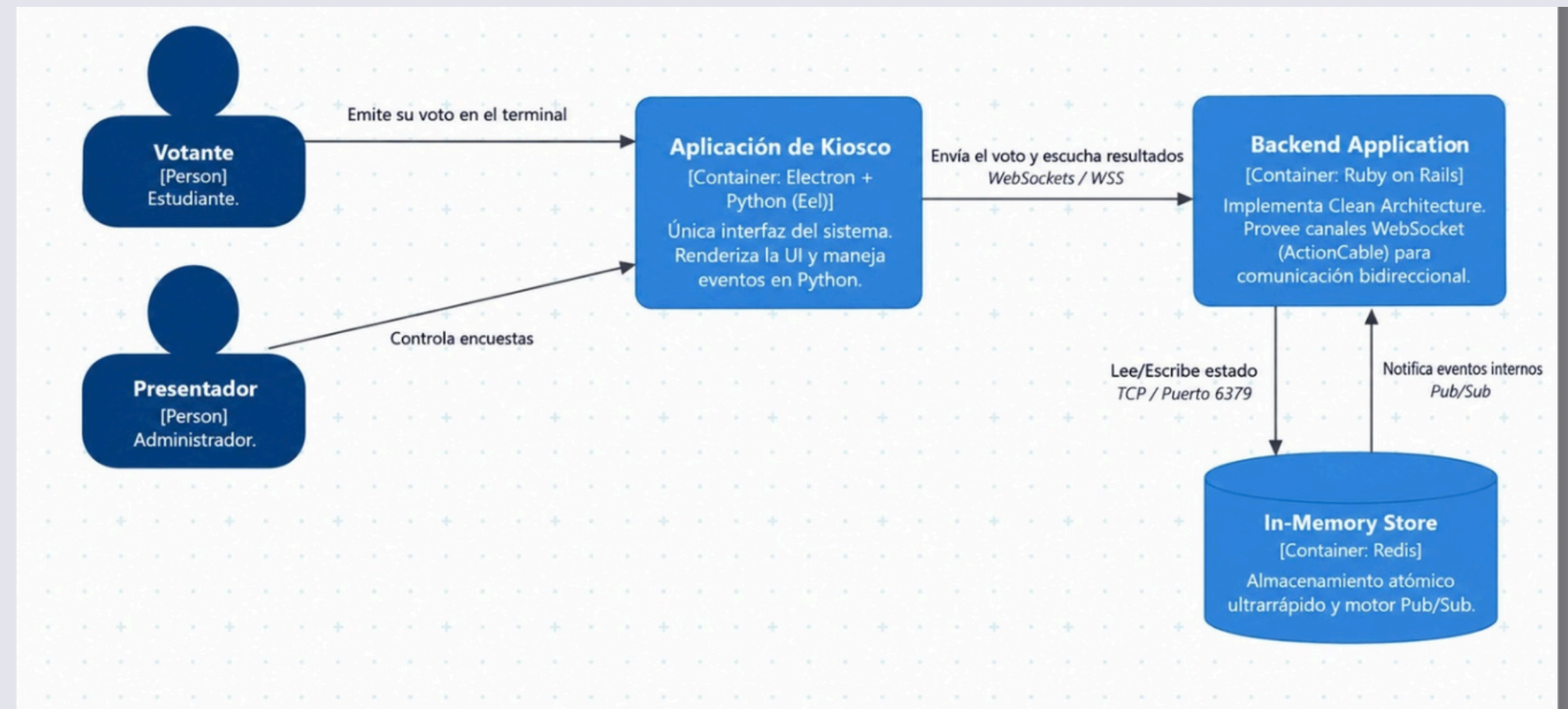
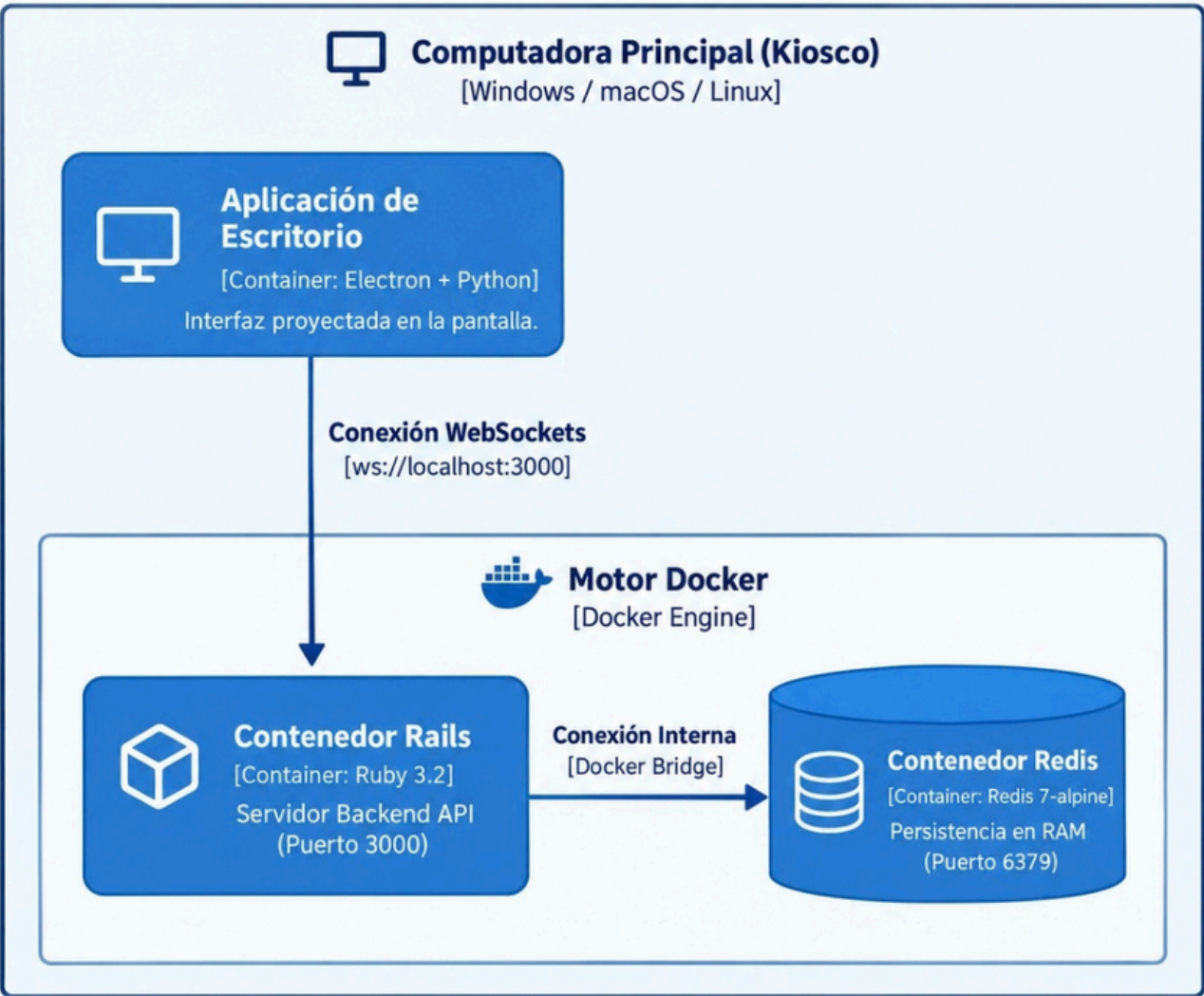


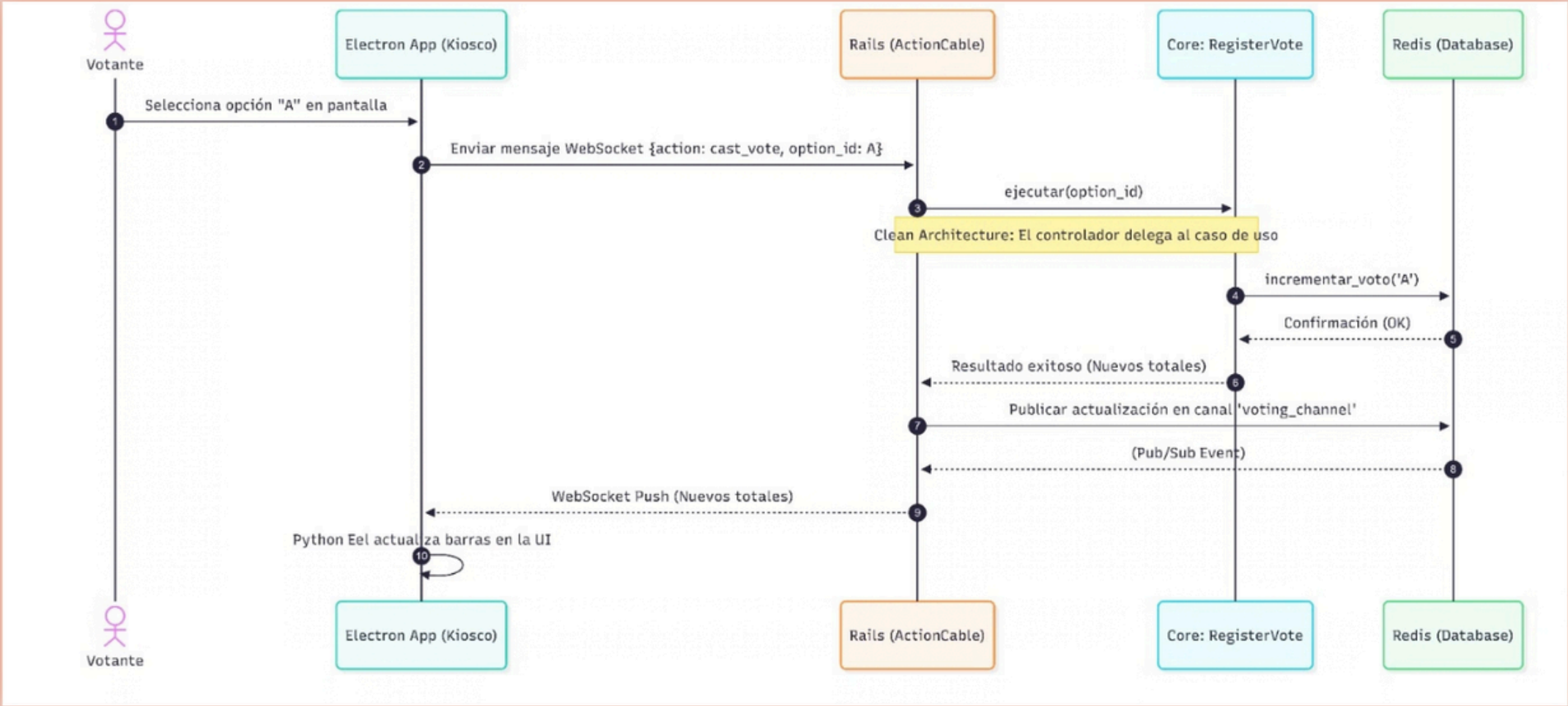
DIAGRAMA DE DESPLIEGUE

Entorno local unificado

Diagrama de Despliegue (Entorno Local Unificado)



FLUJO DINÁMICO DE UN VOTO

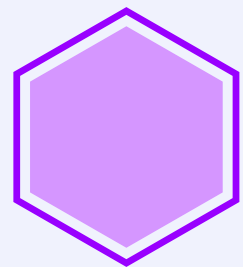


PATRONES

Para el desarrollo de nuestra parte practica se implementaron los siguientes.



Patrón Repositorio (Repository Pattern)



Sirve como un "traductor" o puente entre nuestra lógica de negocio pura y la base de datos (en nuestro caso, Redis).

`redis_vote_repository.rb`

Patrón de Caso de Uso / Interactor (Use Case Pattern)



Encapsula una única acción específica que un usuario puede hacer en el sistema. Es el "Qué" hace la aplicación

`register_vote.rb`

Inyección de Dependencias (Dependency Injection)



Es una regla de Clean Architecture (la "D" de SOLID). Significa que a un objeto no se le permite construir sus propias herramientas, se las pasamos desde afuera.

En el initialize (constructor) de RegisterVote, donde recibe el repository y el broadcaster como parámetros.

Patrón Entidad (Entity Pattern)



Representa los conceptos de negocio puros, independientes de la base de datos o de la web. Son los objetos más estables del sistema.

`app/core/entities/`

ESTRUCTURA **BACKEND**

```
backend/
├─ app/
│  │
│  │ └─ core/
│  │     └─ entities/
│  │         └─ poll.rb
│  │         └─ option.rb
│  │         └─ vote.rb
│  │
│  │ └─ errors.rb
│  │
│  │ └─ use_cases/
│  │     └─ register_vote.rb
│  │     └─ get_poll_results.rb
│  │
│  │ └─ infrastructure/
│  │     └─ repositories/
│  │         └─ redis_vote_repository.rb
│  │
│  │ └─ channels/
│  │     └─ application_cable/
│  │         └─ channel.rb
│  │         └─ connection.rb
│  │         └─ voting_channel.rb
│  │
│  └─ controllers/
│      └─ application_controller.rb
│      └─ polls_controller.rb
```

<— CAPA CENTRAL (DOMINIO Y REGLAS DE NEGOCIO)

<— CAPA 1: ENTIDADES DE DOMINIO (Enterprise Business Rules)

—> Entidad pura PORO: Modela la sesion, titulo y estado (abierta/cerrada)

—> Entidad pura PORO: Modela cada alternativa seleccionable de la mocion

—> Entidad pura PORO: Modela el registro inmutable de auditoria del voto

—> DOMAIN ERRORS: Excepciones de negocio (DuplicateVoteError, PollClosedError)

<— CAPA 2: CASOS DE USO (Application Business Rules)

—> Caso de Uso: Orquesta validacion, persistencia atomica y broadcast

—> Caso de Uso: Orquesta consulta y calculo reactivo de porcentajes

<— CAPA 3: ADAPTADORES DE INFRAESTRUCTURA (Driven / Outbound Adapters)

—> Adaptador de Persistencia: Implementa operaciones atomicas (SADD, HINCRBY)

<— CAPA 3: ADAPTADORES DE ENTREGA PUSH (Driving / Inbound Adapters)

—> Base de canales de ActionCable

—> Identificacion y ciclo de vida de la conexion WebSocket

—> Adaptador WebSocket: Conecta eventos de red con Casos de Uso

<— CAPA 3: ADAPTADORES DE ENTREGA HTTP (Driving / Inbound Adapters)

—> Controlador API base sin vistas HTML

—> Adaptador REST: Endpoints de consulta y contingencia (Fallback HTTP)

ESTRUCTURA **BACKEND**

└─ config/	← CAPA 4: FRAMEWORKS Y DRIVERS (Herramientas y Configuración Externa)
└─ routes.rb	→ Enrutador de Rails y montaje de ActionCable en /cable
└─ puma.rb	→ Servidor web concurrente enlazado a 0.0.0.0:3000
└─ cable.yml	→ Adaptador pub/sub de ActionCable con Redis
└─ initializers/	
└─ clean_architecture.rb	→ Carga determinística de POROs ignorando Zeitwerk
└─ redis.rb	→ Driver cliente Redis y seeding de datos iniciales
└─ cors.rb	→ Política de acceso cruzado HTTP (CORS)
└─ Dockerfile	→ Contenedor de ejecución: Entorno aislado en Alpine Linux
└─ Gemfile	→ Manifiesto de dependencias (Rails 7 API, sin ActiveRecord)

ESTRUCTURA **BACKEND**

Capa de Clean Architecture	Directorio / Archivo	Rol en el Patron de Puertos y Adaptadores
Capa 1: Entidades	<code>app/core/entities/*.rb</code> <code>app/core/errors.rb</code>	Nucleo del Dominio: Objetos puros sin dependencias de base de datos ni frameworks. Reglas de negocio intrinsecas.
Capa 2: Casos de Uso	<code>app/core/use_cases/*.rb</code>	Puertos de Entrada: Clases de proposito unico (<code>RegisterVote</code> , <code>GetPollResults</code>) que dirigen el flujo del sistema.
Capa 3: Adaptadores (Salida)	<code>app/infrastructure/repositories/*.rb</code>	Adaptador de Persistencia (Driven): Satisface el contrato de acceso a datos mediante primitivas en memoria de Redis.
Capa 3: Adaptadores (Entrada)	<code>app/channels/voting_channel.rb</code> <code>app/controllers/polls_controller.rb</code>	Adaptadores de Entrega (Driving): Reciben estímulos externos (WebSockets y HTTP REST) y los traducen a los Casos de Uso.
Capa 4: Frameworks y Drivers	<code>config/*</code> , <code>Dockerfile</code> , Puma	Mecanismos Externos: Herramientas de ejecucion, red y servidor reemplazables sin afectar el negocio.

ESTRUCTURA FRONTEND

```
frontend_desktop/
├── main.py                                ← CONTROLADOR Y ADAPTADOR PRINCIPAL DE ESCRITORIO
│   ├── [Subproceso asyncio (WebSockets)] → Adaptador de Red (Inbound/Outbound):
│   │   ├── Mantiene conexion TCP duplex persistente con ActionCable (/cable)
│   │   ├── Handshake de bienvenida      Recibe 'welcome' y envia suscripcion a 'VotingChannel'
│   │   ├── Bucle de lectura de eventos  Recibe 'results_updated', 'vote_accepted' y despacha a la UI
│   │   └── Reconexion automatica continua Reintenta conexion en caso de corte de red sin cerrar la app
│   ├── [Cola Concurrente (asyncio.Queue)] → Desacoplamiento de Hilos:
│   │   └── outbound_queue                Transmite votos desde el hilo sincrono de Eel hacia el socket asincrono
│   ├── [Consultas Directas HTTP Fallback] → Adaptador de Precarga:
│   │   └── fetch_initial_poll_data()     Consulta /polls/1 para renderizar el estado inicial sin latencia
│   └── [Funciones Puente (@eel.expose)] → Puerto de Comunicacion Inter-Proceso:
│       ├── send_vote(poll_id, option_id) Expone envio de voto desde JavaScript hacia Python
│       ├── reset_session(poll_id)        Expone orden de reinicio desde JavaScript hacia Python
│       └── request_initial_state()        Expone sincronizacion de terminal ID y estado general
├── web/                                  ← CAPA DE PRESENTACION VISUAL (GUI LOCAL EN BROWSER ENGINE)
│   ├── index.html                       → Vista Declarativa (DOM Semantico):
│   │   └── Badges de conexion/nodo, tarjeta de sesion, barras de progreso y botones
│   ├── css/
│   │   └── styles.css                   → Sistema de Diseño y Estado Visual:
│   │       └── Paleta Dark Mode, contrastes semanticos y animaciones reactivas de barras
│   └── js/
│       └── app.js                       → ViewModel / Controlador de Vista en el Cliente:
│           ├── Funciones Expuestas a Python Recibe broadcasts del socket y actualiza el DOM en tiempo real
│           ├── applyTotals()               Calcula ancho de barras y porcentajes dinamicos
│           ├── onVoteClick()              Gestiona eventos del usuario y bloquea doble clic en frontend
│           └── disableVoteButtons()        Preserva texto original de opciones y deshabilita botones post-voto
```

ESTRUCTURA FRONTEND

Componente del Frontend	Archivo / Modulo	Responsabilidad Arquitectonica
Adaptador de Red (WebSocket Client)	<code>main.py</code> (<code>websocket_listener</code>)	Gestiona el protocolo TCP persistente con ActionCable, deserializa payloads JSON y tolera fallos de red.
Adaptador Inter-Capa (Eel Bridge)	<code>main.py</code> (<code>@eel.expose</code>)	Puertos y adaptadores locales: permite la invocacion segura entre el runtime de Python y el motor de JavaScript.
Controlador de Vista (ViewModel)	<code>web/js/app.js</code>	Mantiene el estado local de la terminal (<code>hasVotedInSession</code>), maneja la reactividad de las barras y previene el fraude en el cliente.
Vista Pura (UI)	<code>web/index.html</code> <code>web/css/styles.css</code>	Capa mas externa de presentacion: estructura visual desacoplada que puede ser reemplazada sin tocar la logica de Python.
Driver de Empaquetado	<code>build_executable.bat</code>	Empaqueta el cliente en un ejecutable monolitico de escritorio para distribucion sin dependencias locales.

PARTES CLAVE

CLEAN ARCHITECTURE: REGLAS DE NEGOCIO AISLADAS

```
# backend/app/core/use_cases/register_vote.rb
def execute(poll_id:, option_id:, voter_fingerprint:)

  # 1. Validar regla: ¿Existe la votación?
  poll = @repository.find_poll(poll_id)
  raise Core::Errors::PollNotFoundError unless poll

  # 2. Validar regla: ¿Está abierta la sesión?
  raise Core::Errors::PollClosedError unless poll.open?

  # 3. Guardar usando una abstracción (No tocamos bases de datos
  directamente aquí)
  results = @repository.record_vote(
    poll_id: poll_id,
    option_id: option_id,
    voter_fingerprint: voter_fingerprint
  )

  # 4. Transmitir el evento en tiempo real
  @broadcaster.broadcast("voting_session", results)
end
```

PARTES CLAVE

FRONTEND DESKTOP: TÚNEL DE WEBSOCKETS

```
# frontend_desktop/main.py
import websockets
import json
import eel

async def connect_to_rails():
    # Abrimos el túnel constante con el servidor
    async with websockets.connect("ws://127.0.0.1:3000/cable") as ws:

        while True:
            # Escuchamos pasivamente los gritos del servidor
            raw_msg = await ws.recv()
            payload = json.loads(raw_msg)

            if payload.get("message", {}).get("event") ==
"results_updated":
                # Al recibir nuevos resultados, actualizamos la gráfica
HTML
                eel.update_poll_display(payload["message"]["totals"])()
```

INFRAESTRUCTURA: CONTEO ATÓMICO EN MEMORIA (REDIS)

```
# backend/app/infrastructure/repositories/redis_vote_repository.rb
def record_vote(poll_id:, option_id:, voter_fingerprint:)

    # Operación Atómica 1: Prevenir Fraude en tiempo O(1)
    is_new = @redis.sadd("poll:#{poll_id}:voters", voter_fingerprint)
    if is_new == 0
        raise Core::Errors::DuplicateVoteError, "¡El delegado ya emitió su
voto!"
    end

    # Operación Atómica 2: Sumar el voto en memoria RAM
    @redis.hincrby("poll:#{poll_id}:options", option_id, 1)

    calculate_results(poll_id)
end
```

DEMOSTRACIÓN EN VIVO



LECCIONES APRENDIDAS

Aplicar Clean Architecture en Rails fue duro al principio (Rails prefiere el acoplamiento), pero facilitó enormemente probar la lógica sin depender de la base de datos.

Lenguajes Diferentes, Un Solo Idioma: Conectar Python y Ruby nos obligó a crear contratos JSON muy estrictos por WebSockets.

El Poder de Redis: Aprendimos que hacer conteos con comandos atómicos en memoria es imbatible en velocidad, aunque nos obligó a diseñar las "llaves" manualmente.

Al enfocarnos en un "Kiosco de Escritorio" en lugar de algo mixto y gigante, cumplimos las restricciones y logramos un sistema estable.

MUCHAS GRACIAS